

# Robot Actuator Loop

Single-threaded UDP control loop · 9 actuators · 1kHz · p99 < 200μs

**10.2M**

Total Measurements

**5 × 5 min**

Benchmark Runs

**94.4μs**

Mean p99 (corrected)

# System Overview

## What It Is

### Two C programs over UDP on localhost:

**actuator.c:** simulates one robot joint. Listens on its own UDP port, replies with a sine-wave position ( $180 \cdot \sin(2\pi t)$ ).

**orchestrator.c:** single-threaded control loop. Sends commands to all 9 actuators every 1ms (fast tick). Every 1 second, appends the last 1000 commands to JSON on disk (slow tick).

**Message format:** 20-byte struct: `actuator_id`, `counter`, `timestamp_ns`, `position`.

## The Engineering Challenge

### Single-thread constraint:

No threads, no subprocesses. Fast tick and slow tick both run on one OS thread.

### Latency deadline:

p99 RTT must stay under 200 $\mu$ s even while disk I/O periodically occurs.

### Bus architecture:

3 buses  $\times$  3 actuators = 9 total. Each gets a unique port (`PORT_BASE + bus $\times$ 3 + id`) to avoid routing ambiguity.

# Timing Methodology

## RTT Definition

`send_times[i] = now_ns()` stamped just before `sendto()`.  
RTT recorded when `recvfrom()` returns the response. Uses `mach_absolute_time()` on Mac for nanosecond-precision monotonic clock.

## Non-Blocking Sockets

`fcntl(sock, F_SETFL, O_NONBLOCK)` on each of 9 sockets.  
`recvfrom()` returns instantly with or without data. Poll loop spins over all 9 sockets until all respond or the 0.9ms deadline passes.

## Benchmark Setup

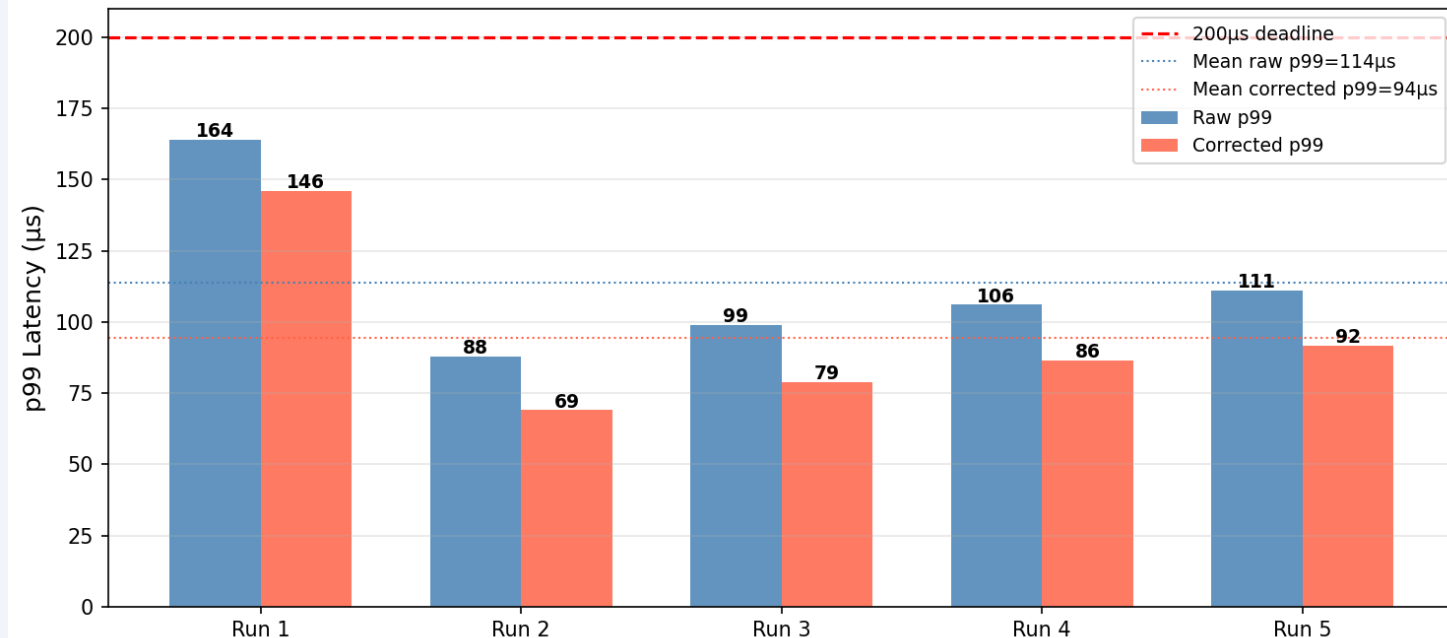
5 independent runs × 5 minutes = 10,166,040 total measurements. Each row records: counter, bus, target\_id, target\_idx (send order 0-8), rtt\_us.

## Send-Offset Correction

Sequential sends create a gradient in raw RTTs (resulting in additional time for earlier targets). Correction:  $\text{overhead} = (\text{avg\_idx0} - \text{avg\_idx8}) / 8$ .  $\text{corrected} = \text{raw} - ((8 - \text{idx}) \times \text{overhead})$ .

# Results - p99 Latency per Run

p99 Latency Per Run — Raw vs Corrected  
(9 Actuators, 3 Buses, 1kHz, 5 min each)



**113.6µs**

Mean raw p99

**94.4µs**

Mean corrected p99

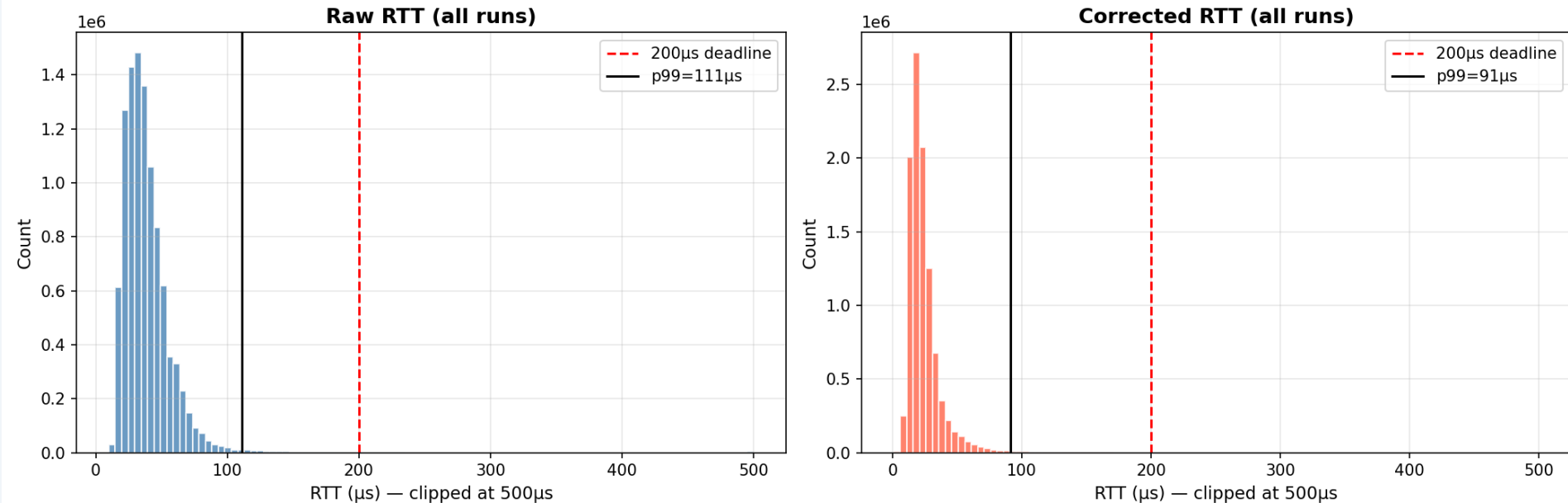
**< 200µs**

All 5 runs pass deadline

*Run 1 outlier (164µs p99, 390ms max): background OS interference / waking from sleep. All 5 runs meet 200µs deadline.*

# Results - Latency Distribution

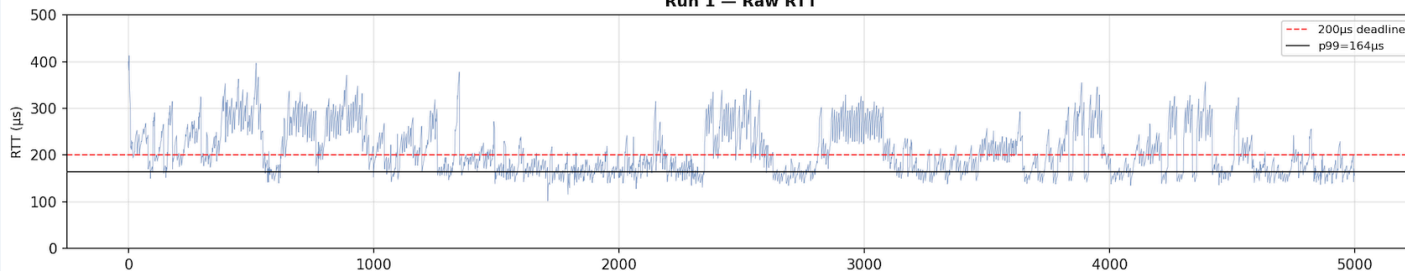
Aggregate RTT Distribution — Raw vs Corrected (5 × 5min Runs)



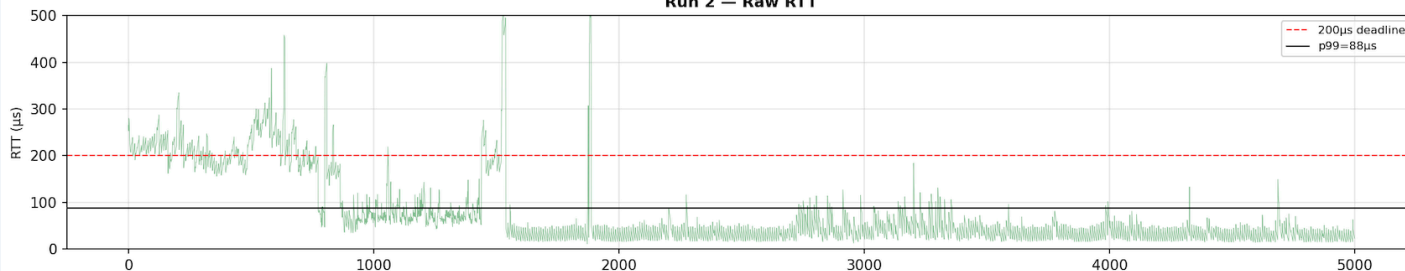
**Tight cluster (10-50 $\mu\text{s}$ ):** true UDP round-trip + sine computation (system working as intended). **Long right tail:** rare OS scheduler preemptions. Corrected histogram (right) is tighter (send-loop bias removed).

# Results - RTT Over Time (Runs 1-3)

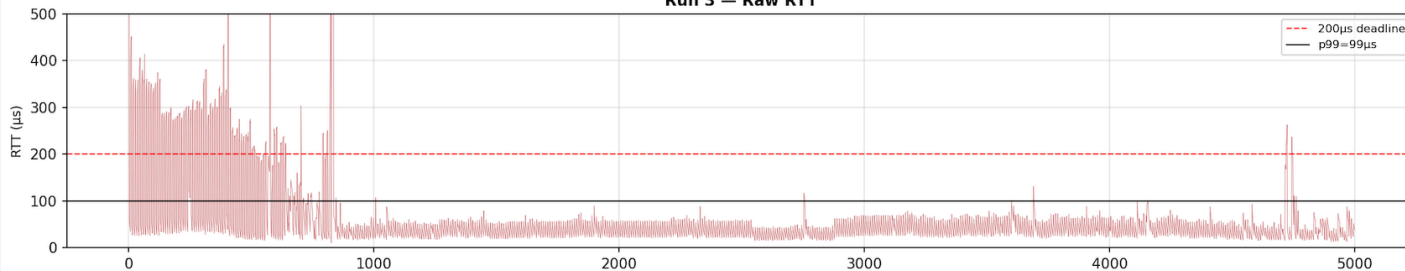
Raw RTT Over Time — All Runs (first 5000 samples)  
Run 1 — Raw RTT



Run 2 — Raw RTT



Run 3 — Raw RTT



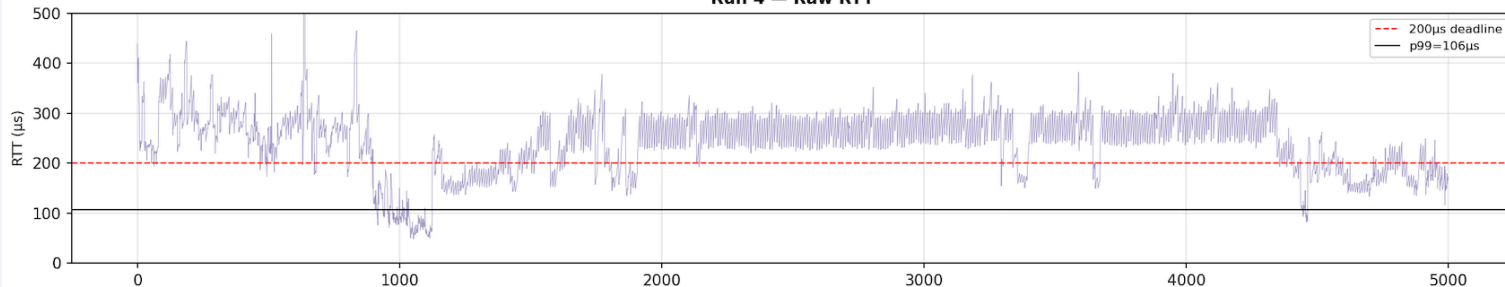
Runs 2, 3, 5: warm-up drop  
~200 $\mu\text{s}$  → 50-100 $\mu\text{s}$ . Cache  
warming, CPU freq scaling,  
page faults.

Run 4: multiple  
transitions - intermittent  
background process  
interference.

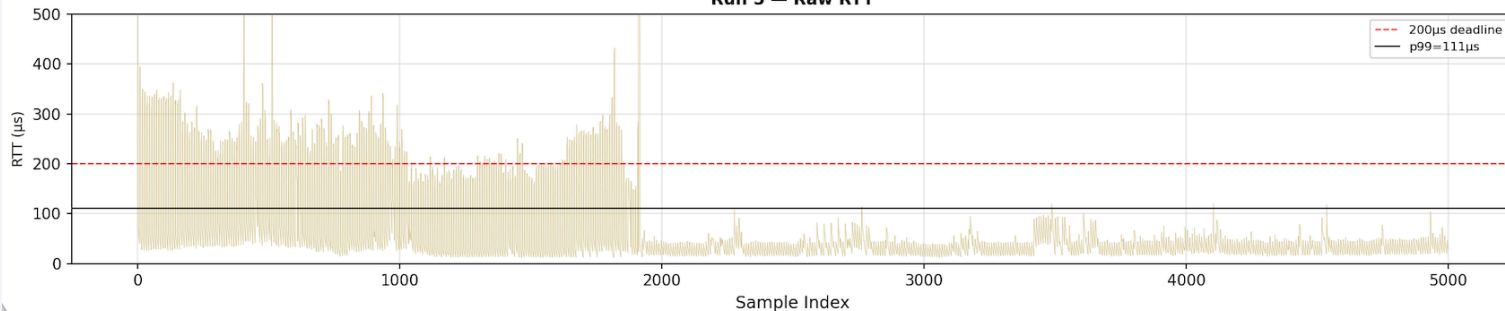
Run 1: no warm-up drop -  
elevated throughout,  
consistent with 390ms  
outlier.

# Results - RTT Over Time (Runs 4-5)

Run 4 — Raw RTT



Run 5 — Raw RTT

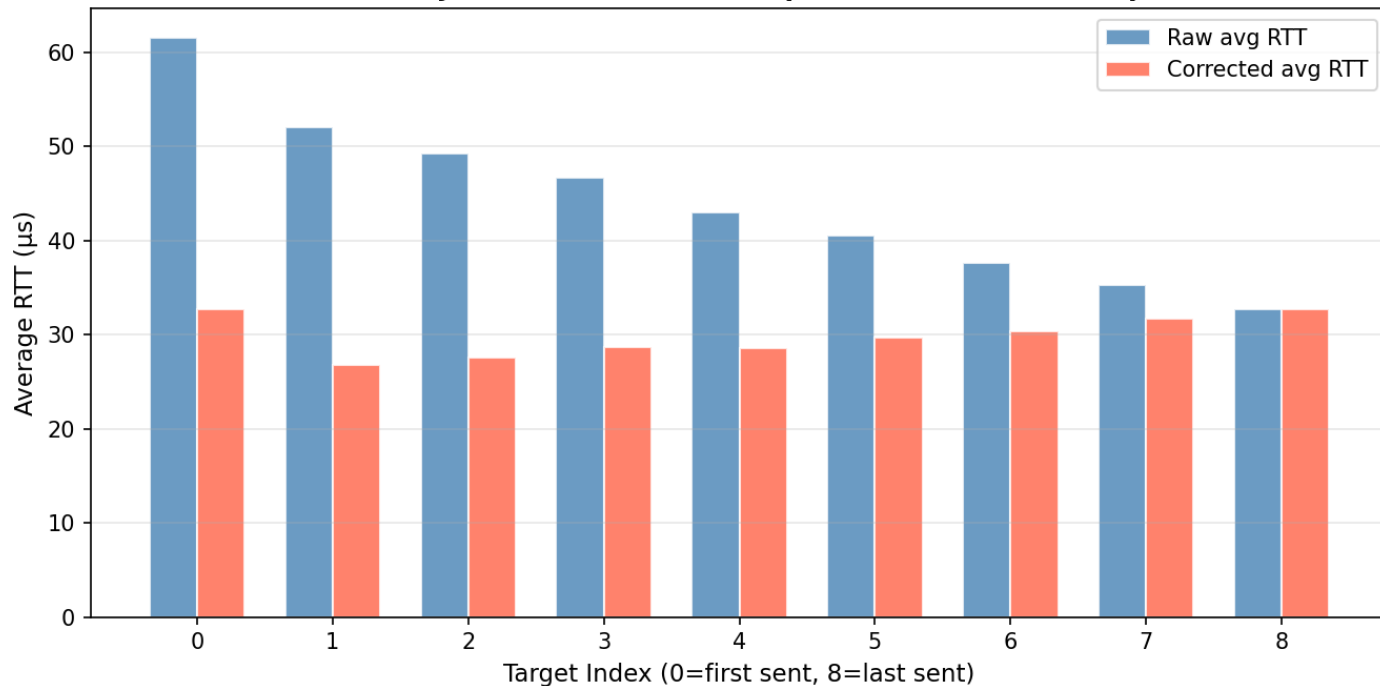


Runs 2, 3, 5: warm-up drop  
~200 $\mu$ s  $\rightarrow$  50-100 $\mu$ s. Cache  
warming, CPU freq scaling,  
page faults. (Runs 2 and 3  
shown on previous slide).

Run 4: multiple  
transitions - intermittent  
background process  
interference.

# Results - Send-Offset Gradient

**Send Offset Gradient — Raw vs Corrected**  
Shows systematic bias from sequential send timestamps



Raw RTT slopes from  
~62μs (idx 0, sent first)  
to ~33μs (idx 8, sent  
last).

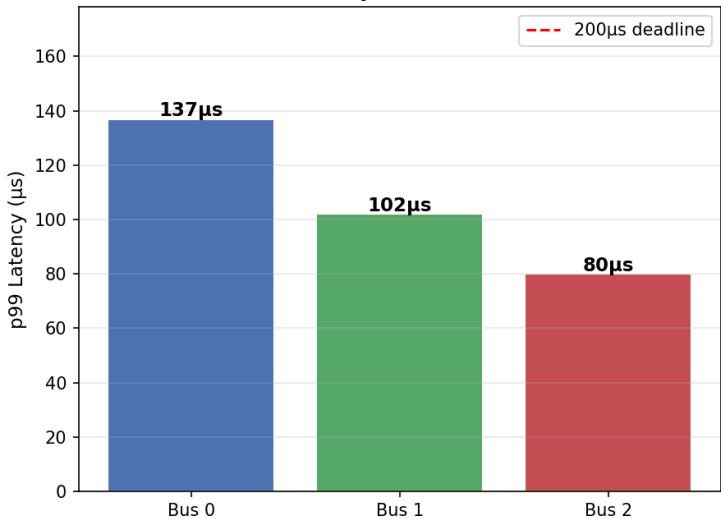
Send-Offset Gradient



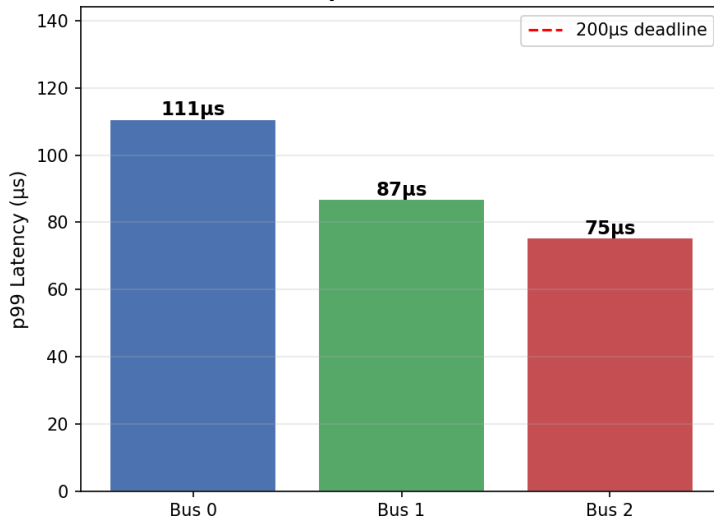
# Results - Per-Bus p99

Per-Bus p99 — Aggregate (5 Runs)

Per-Bus p99 — Raw RTT



Per-Bus p99 — Corrected RTT



Per-Bus p99

Bus 0 (targets 0-2, sent and polled first) shows highest p99 even after correction (send-order bias dominates per-bus tail behavior).

# Key Takeaways

## 01

### Blocking → Non-Blocking was the critical change

p99 dropped from 265μs (failing) to 94-114μs (passing). Blocking `recvfrom()` froze the thread, while non-blocking polling reads whichever responses are ready.

## 02

### Single-thread design holds up by sequencing correctly

RTTs are recorded before `write_log()` runs. By construction, the slow tick cannot inflate that tick's measurement.

## 03

### OS jitter dominates the tail, not the code

Tight cluster (10-50μs) = true round-trip speed. The tail and 390ms outlier are OS scheduling events, which is expected on a general-purpose desktop OS.

## 04

### Next Steps: Using `SO_TIMESTAMP` to remove loop overhead

`SO_TIMESTAMP` captures kernel arrival time, removing receive-loop read-order bias, eliminating loop overhead from being included when measuring RTT.